



IMPLEMENTACIÓN CAN BUS EN VEHÍCULOS

DOCUMENTO TÉCNICO FINAL



Marck D. Carrión Guzmán

KATALYST

Departamento de Ingeniería
Iturmendi, España

10 de septiembre de 2025

Índice

1. Introducción	3
2. Marco Teórico	4
2.1. Protocolo CAN BUS	4
2.1.1. Tramas	6
2.2. Unidades de Control Electrónico (ECUs)	8
2.3. El protocolo ISP (In-System Programming)	8
2.4. Protocolo I ² C (Inter-Integrated Circuit)	10
2.5. Protocolo USART (Universal Synchronous/Asynchronous Receiver-Transmitter)	12
2.6. El MCP2515 como controlador CAN	14
2.7. Microcontroladores y plataformas embebidas utilizadas	14
2.8. Arquitectura distribuida en automoción	15
2.9. Diagnóstico mediante OBD-II	15
3. Objetivos del proyecto	16
3.1. Objetivo general	16
3.2. Objetivos específicos	16
4. Metodología	17
4.1. Fase 1: Análisis inicial	17
4.2. Fase 2: Diseño del sistema	18
4.3. Fase 3: Desarrollo de hardware	18
4.4. Fase 4: Desarrollo de software	19
4.5. Fase 5: Integración con aplicación móvil	19
4.6. Fase 6: Validación experimental	20
5. Diseño y desarrollo del sistema	20
5.1. Arquitectura general del sistema	21
5.2. Diseño de hardware	22
5.2.1. Módulos de prueba	22
5.2.2. Placas personalizadas	22
5.3. Diseño de software	23
5.3.1. ECUs (Raspberry Pi)	23
5.3.2. Nodos secundarios (Atmega)	23
5.3.3. Aplicación móvil	24
5.4. Control por voz mediante inteligencia artificial	24
5.5. Flujo de comunicación	24
5.6. Desarrollo	25

5.6.1. Módulos de temperatura del habitáculo	25
6. Bitácora de Experimentos	29

1. Introducción

La evolución de los sistemas electrónicos en el sector de la automoción ha permitido que los vehículos modernos cuenten con redes de comunicación internas avanzadas, como el protocolo *CAN BUS*, que interconectan múltiples unidades de control electrónico (ECUs) para funciones de confort, seguridad, motorización y multimedia. Sin embargo, muchos vehículos fabricados en décadas anteriores carecen de estas capacidades, lo que limita significativamente su nivel de automatización y conectividad.

Este proyecto surge con el objetivo de transformar un vehículo antiguo —equipado originalmente únicamente con una unidad de control de motor conectada a través del conector OBD para diagnóstico— en una plataforma más avanzada, dotada de nuevas funcionalidades que permitan ampliar su experiencia de uso y control interno.

Para ello, se propone la integración de un sistema de automatización distribuido basado en el protocolo *CAN BUS*, empleando controladores diseñados por el propio equipo de desarrollo. Estos controladores estarán basados en el chip **MCP2515**, un controlador CAN SPI ampliamente utilizado, que permitirá establecer nodos adicionales dentro de la red del vehículo.

El proyecto contempla la creación de al menos dos nuevas unidades funcionales:

- **Unidad de confort:** encargada de funciones como el control de climatización, cierre centralizado, control de ventanas y posibles ajustes de iluminación interior.
- **Unidad multimedia:** enfocada en la gestión de audio, interfaces con el usuario,

conectividad y notificaciones del sistema.

Estas unidades estarán interconectadas mediante un nuevo bus CAN estructurado, que permitirá comunicación en tiempo real y expansión futura. El sistema será desarrollado, probado e integrado completamente por el equipo, aprovechando tanto hardware propio como soluciones abiertas adaptadas al entorno del vehículo.

Con esta iniciativa se busca no solo modernizar un automóvil existente, sino también explorar las posibilidades de extender tecnologías de automoción a plataformas donde originalmente no estaban concebidas, lo cual abre nuevas puertas para la actualización tecnológica de flotas antiguas o proyectos de restauración inteligente.

2. Marco Teórico

En esta sección se presentan los fundamentos técnicos y conceptuales que sustentan el desarrollo del presente proyecto. Se abordan tecnologías clave del sector automotriz, enfocadas principalmente en la comunicación interna entre sistemas electrónicos.

2.1. Protocolo CAN BUS

El **Controller Area Network** (CAN BUS) es un protocolo de comunicación serial creado por Bosch en los años 80, diseñado para aplicaciones en la automoción y estandarizado posteriormente en la norma ISO 11898 [**iso11898**]. Su característica esencial es permitir que múltiples microcontroladores y dispositivos (nodos) se comuniquen entre sí sin necesidad de un maestro central.

La robustez del protocolo se debe a su esquema de arbitraje: cuando dos nodos transmiten al mismo tiempo, prevalece el mensaje con identificador de menor valor, lo que asegura una comunicación determinista y priorizada. Además, utiliza comunicación diferencial mediante dos líneas (CAN_H y CAN_L), alcanzando velocidades típicas de hasta 1 Mbps.

El bus CAN utiliza un protocolo **basado en mensajes**, donde los datos se transmiten en tramas en lugar de conexiones punto a punto.

Cada trama de mensaje CAN consta de varios campos: el **Inicio de Trama** (SOF), un Identificador (ID), Campo de Control, Campo de Datos, **Verificación por Redundancia Cíclica** (CRC), Campo de Reconocimiento (ACK) y el Fin de Trama (EOF). El Identificador es crucial, ya que determina la prioridad del mensaje; los valores numéricos más bajos tienen mayor prioridad. Esto permite que los mensajes críticos, como los relacionados con el frenado o el control del motor, se transmitan antes que datos menos urgentes.

El protocolo CAN utiliza un mecanismo de Acceso Múltiple por **Detección de Portadora con Detección de Colisiones** (CSMA/CD), pero con un proceso de arbitraje único. Cuando varios nodos intentan transmitir al mismo tiempo, el nodo con el ID más bajo gana la arbitraje y continúa transmitiendo, mientras que los demás se retiran automáticamente, lo que garantiza que no se pierda ningún dato.

2.1.1. Tramas

El bus Controller Area Network (CAN) utiliza una estructura específica de tramas de mensajes para la transmisión de datos, lo que garantiza una comunicación eficiente y confiable entre las unidades de control electrónico (ECUs) en entornos como los sistemas automotrices e industriales. El protocolo CAN define varios tipos de tramas: **tramas de datos, tramas remotas, tramas de error y tramas de sobrecarga**, cada una con una función específica.

Trama de datos

Una trama de datos se utiliza para transmitir información desde un nodo a uno o más nodos receptores y consta de varios campos. Comienza con el Inicio de Trama (**SOF**), un solo bit dominante (lógica 0) que señala el comienzo de un nuevo mensaje y sincroniza todos los nodos en el bus. Después del SOF, sigue el **Campo de Arbitraje**, que contiene el ID del mensaje (11 bits en formato estándar, permitiendo 2048 identificadores únicos) y el bit **Remote Transmission Request (RTR)**. El bit RTR distingue entre una trama de datos (RTR = 0) y una trama remota (RTR = 1). El ID del mensaje también determina la prioridad del mensaje durante la arbitraje del bus; **un ID numéricamente más bajo tiene mayor prioridad.**

A continuación se encuentra el Campo de Control, que incluye bits reservados (R0, R1) y el **Código de Longitud de Datos (DLC)**, un campo de 4 bits que indica la cantidad de bytes de datos (de 0 a 8) en la trama. Le sigue el Campo de Datos, que contiene la carga útil del mensaje, con un máximo de 8 bytes en el CAN estándar. Después de los

datos, un campo de **Verificación por Redundancia Cíclica (CRC)** de 15 bits garantiza la detección de errores; el receptor calcula su propio CRC y lo compara con el valor transmitido. Una discrepancia indica un error de transmisión.

El **Campo de Reconocimiento (ACK)** consta de una ranura ACK y un delimitador ACK. El nodo transmisor envía un bit recesivo en la ranura ACK, que los nodos receptores sobrescriben con un bit dominante (lógica 0) para confirmar la recepción exitosa; si esto no ocurre, se produce un error de ACK. La trama concluye con el **Fin de Trama (EOF)**, siete bits recesivos que marcan el final de la trama, seguidos por un espacio de Intermisión (IFS) de 3 bits, tras lo cual el bus vuelve al estado inactivo y cualquier nodo puede iniciar una nueva transmisión.

Tramas remotas

Las tramas remotas son estructuralmente similares a las tramas de datos, pero carecen del campo de datos; se utilizan para que un nodo solicite datos a otro nodo, que luego responde con una trama de datos usando el mismo ID de mensaje.

Tramas de error

Las tramas de error son transmitidas por cualquier nodo que detecta un error y sirven para abortar la trama actual, mientras que las tramas de sobrecarga introducen una demora entre tramas para gestionar la carga de procesamiento de los nodos.

2.2. Unidades de Control Electrónico (ECUs)

Las ECUs son microcontroladores dedicados al control de funciones específicas dentro del vehículo, como el motor, la transmisión, los frenos, la climatización o el sistema de infoentretenimiento. Cada ECU intercambia información con otras a través del bus CAN, conformando una arquitectura de red interna.

En automóviles modernos, el número de ECUs puede superar las 50 unidades, reflejando la creciente complejidad electrónica. En contraste, en vehículos más antiguos las funciones electrónicas eran más aisladas y, en muchos casos, limitadas al diagnóstico del motor.

2.3. El protocolo ISP (In-System Programming)

El ISP (*In-System Programming*) es un protocolo de programación que permite cargar firmware en microcontroladores directamente en la placa donde se encuentran montados, sin necesidad de retirarlos o emplear programadores externos especializados. Este enfoque resulta fundamental en el desarrollo de sistemas embebidos, ya que facilita la iteración rápida, la actualización de software y el mantenimiento de los dispositivos una vez desplegados en el entorno de aplicación.

El principio básico de ISP consiste en habilitar al microcontrolador para que, mediante una interfaz de comunicación serial dedicada, reciba instrucciones de programación desde un dispositivo externo (generalmente un computador o un programador USB) y almacene el código binario en su memoria flash. Para ello, el microcontrolador dispone de un **bootloader** o de rutinas internas específicas que gestionan este proceso.

Características principales

- **Acceso directo a la memoria:** permite escribir y reescribir el contenido de la memoria de programa sin desmontar el microcontrolador.
- **Uso de pines estándar:** típicamente se emplean líneas como MOSI, MISO, SCK y RESET, compartidas con la interfaz SPI del dispositivo.
- **Compatibilidad amplia:** la mayoría de los microcontroladores de las familias Atmel AVR, PIC y algunos ARM Cortex incluyen soporte para ISP.
- **Velocidad de programación:** el proceso es relativamente rápido debido a la naturaleza serial síncrona de la comunicación.

Aplicaciones en el proyecto

En el contexto del presente trabajo, el protocolo ISP será utilizado para programar los microcontroladores **Atmega** que actúan como nodos en la red CAN. Estos microcontroladores, equipados con el chip MCP2515 para conectividad, requieren que su firmware se cargue y actualice durante el ciclo de desarrollo y pruebas. La posibilidad de reprogramarlos en la propia tarjeta sin desoldar ni extraer componentes representa una ventaja significativa en términos de agilidad y reducción de costos.

Asimismo, el uso de ISP permite realizar ajustes incrementales en los algoritmos de interpretación de mensajes CAN, control de sensores o activación de actuadores, garantizando un flujo de desarrollo iterativo y flexible. Esta capacidad resulta clave en proyectos de I+D donde la experimentación y la mejora continua son parte inherente del proceso.

2.4. Protocolo SPI (Serial Peripheral Interface)

El **SPI** (*Serial Peripheral Interface*) es un protocolo de comunicación serial síncrono desarrollado por Motorola a mediados de los años 80, ampliamente utilizado en sistemas embebidos para la interconexión de microcontroladores con periféricos de alta velocidad, tales como memorias, sensores, convertidores A/D y controladores de comunicación como el **MCP2515** utilizado en este proyecto. Su diseño prioriza la **velocidad y simplicidad**, lo que lo convierte en una de las interfaces más empleadas en aplicaciones que requieren intercambio rápido y confiable de datos.

Principios de funcionamiento

El protocolo SPI se basa en un esquema **maestro-esclavo**, donde un dispositivo maestro controla el flujo de comunicación mediante una señal de reloj compartida. Para ello, utiliza cuatro líneas principales:

- **MOSI (Master Out, Slave In)**: línea de datos desde el maestro hacia el esclavo.
- **MISO (Master In, Slave Out)**: línea de datos desde el esclavo hacia el maestro.
- **SCK (Serial Clock)**: señal de reloj generada por el maestro para sincronizar la transferencia.
- **SS (Slave Select)**: señal para habilitar o deshabilitar un esclavo específico en el bus.

La transferencia de datos se realiza de forma **full-duplex**, lo que significa que mientras el maestro envía un bit por MOSI, simultáneamente recibe otro desde MISO. Esto permite velocidades de comunicación superiores a las de protocolos como I²C o UART.

Características principales

- **Alta velocidad de transferencia:** puede alcanzar decenas de MHz en implementaciones típicas.
- **Simplicidad de protocolo:** no requiere direccionamiento complejo ni tramas extensas.
- **Full-duplex:** permite transmisión y recepción simultánea de datos.
- **Flexibilidad en configuración:** parámetros como polaridad y fase del reloj (CPOL y CPHA) pueden ajustarse para adaptarse a distintos periféricos.
- **Limitación en número de esclavos:** cada dispositivo requiere su propia línea de selección (SS), lo que dificulta la escalabilidad en sistemas con muchos periféricos.

Aplicaciones en el proyecto

En el presente proyecto, el protocolo SPI se emplea como interfaz de comunicación entre los microcontroladores **Atmega** y el controlador CAN **MCP2515**. Gracias a la velocidad de SPI, es posible enviar y recibir tramas CAN de forma eficiente, asegurando que los nodos diseñados puedan integrarse correctamente en la red del vehículo.

Además, el uso de SPI permite que los microcontroladores intercambien información con otros periféricos de apoyo, como memorias externas o sensores de alta velocidad, en caso de ser necesario. De esta forma, SPI actúa como un **punto fundamental** entre la lógica de los microcontroladores y el hardware de comunicación que conecta al vehículo con la red CAN distribuida.

2.5. Protocolo I²C (Inter-Integrated Circuit)

El protocolo **I²C** (*Inter-Integrated Circuit*) es un estándar de comunicación serial desarrollado por Philips a principios de los años 80, diseñado para permitir la interconexión sencilla de múltiples dispositivos electrónicos empleando únicamente dos líneas de comunicación: **SDA** (Serial Data) y **SCL** (Serial Clock). Su filosofía se centra en la simplicidad de conexión, bajo consumo de pines y capacidad de escalar a configuraciones maestro-esclavo en sistemas embebidos.

Principios de funcionamiento

El bus I²C permite que un **dispositivo maestro** controle la comunicación, generando las señales de reloj (SCL) y enviando las instrucciones a uno o más **dispositivos esclavos**. Cada esclavo conectado al bus dispone de una dirección única, lo que hace posible que múltiples circuitos integrados (sensores, memorias, conversores A/D, etc.) compartan las mismas líneas físicas sin interferencia.

El protocolo es síncrono y soporta velocidades de transferencia que van desde:

- 100 kHz (modo estándar),
- 400 kHz (modo rápido),
- hasta varios MHz en implementaciones extendidas (Fast-mode Plus, High-speed mode).

Características principales

- **Uso eficiente de pines:** solo requiere dos líneas para interconectar múltiples dispositivos.

- **Escalabilidad:** admite múltiples esclavos y, en algunas implementaciones, múltiples maestros.
- **Direccionamiento:** cada esclavo posee una dirección de 7 o 10 bits, lo que posibilita la conexión de decenas de periféricos en un mismo bus.
- **Comunicación half-duplex:** en la misma línea SDA se transmiten y reciben datos, bajo control del maestro.
- **Soporte en hardware:** gran variedad de microcontroladores incluyen módulos I²C dedicados.

Aplicaciones en el proyecto

En el presente proyecto, el protocolo I²C se considera clave para la **interconexión local de sensores** dentro de cada nodo controlado por microcontroladores Atmega o Raspberry Pi. Por ejemplo, los sensores de **temperatura, humedad y presión ambiental** suelen disponer de interfaces I²C, lo que simplifica su integración al requerir solo dos pines del microcontrolador.

De este modo, mientras que el **CAN BUS** sirve como red troncal de comunicación entre las distintas ECUs del vehículo, el **I²C** funciona como un sub-bus interno dentro de cada ECU, facilitando la conexión de múltiples periféricos sin necesidad de recurrir a buses adicionales más complejos. Esta complementariedad entre ambos protocolos asegura un diseño modular y escalable del sistema.

2.6. Protocolo USART (Universal Synchronous/Asynchronous Receiver-Transmitter)

El **USART** (*Universal Synchronous/Asynchronous Receiver-Transmitter*) es un módulo de comunicación serial integrado en numerosos microcontroladores, cuya función es transmitir y recibir datos de manera eficiente mediante una interfaz configurable tanto en modo síncrono como en modo asíncrono. A diferencia de buses como I²C o SPI, el USART no define un protocolo completo de interconexión, sino que proporciona una **interfaz física y lógica** para la transmisión de datos serializados entre dispositivos.

Principios de funcionamiento

En su modo más común, el **asíncrono**, el USART implementa la comunicación serial UART tradicional, donde los datos se transmiten bit a bit a través de dos líneas:

- **TX (Transmit):** línea de transmisión de datos.
- **RX (Receive):** línea de recepción de datos.

El flujo de datos está organizado en **tramas** que incluyen bits de inicio, bits de datos, opcionalmente un bit de paridad y uno o más bits de parada. Esta estructura permite que el receptor sincronice la comunicación sin necesidad de compartir una señal de reloj, aunque ambas partes deben configurarse a la misma velocidad de transmisión (*baud rate*).

En el modo **síncrono**, el USART emplea además una línea de reloj compartida que asegura la sincronización exacta entre transmisor y receptor, lo que permite mayores velocidades de transferencia y una comunicación más robusta.

Características principales

- **Flexibilidad:** soporta tanto transmisión síncrona como asíncrona.
- **Velocidad:** puede operar en un amplio rango de tasas de transmisión, típicamente entre 300 baudios y varios Mbps.
- **Eficiencia:** requiere solo dos líneas en modo asíncrono (TX y RX), y tres en modo síncrono (TX, RX y CLK).
- **Compatibilidad:** base de estándares de comunicación ampliamente usados como RS-232, RS-485 y TTL serial.
- **Implementación simple:** disponible de forma nativa en la mayoría de microcontroladores.

Aplicaciones en el proyecto

En este proyecto, el uso del protocolo USART se contempla principalmente para:

- **Depuración y pruebas:** permitir la comunicación entre los microcontroladores (Atmega o Raspberry Pi) y un computador externo para monitorear mensajes, estados del bus CAN y parámetros de sensores.
- **Configuración inicial:** facilitar la carga de parámetros de red o ajustes de firmware durante la fase de pruebas en banco.
- **Interconexión de módulos externos:** algunos periféricos que no soportan I²C pueden integrarse mediante interfaces UART, garantizando versatilidad en la expansión del sistema.

Gracias a su simplicidad y soporte extendido, el USART constituye una herramienta esencial en el ciclo de desarrollo, complementando al bus CAN (para comunicación distribuida en el vehículo) y a I²C (para la integración de sensores locales en cada nodo).

2.7. El MCP2515 como controlador CAN

Para integrar nodos CAN en sistemas embebidos que no cuentan con este protocolo de manera nativa, se emplean controladores externos como el **MCP2515**, fabricado por Microchip Technology. Este chip, conectado mediante interfaz SPI, implementa el protocolo CAN 2.0B y facilita la transmisión y recepción de mensajes.

Entre sus características destacan la capacidad de trabajar a 1 Mbps, tres buffers de transmisión y dos de recepción, así como filtros y máscaras programables que permiten gestionar eficientemente el tráfico en el bus. En este proyecto, el MCP2515 actúa como intermediario entre los microcontroladores y el bus CAN, garantizando escalabilidad y compatibilidad con estándares de la industria.

2.8. Microcontroladores y plataformas embebidas utilizadas

El proyecto contempla el uso de microcontroladores **Atmega** (como los empleados en placas Arduino) para funciones de control en tiempo real y bajo consumo, en conjunto con plataformas más potentes como **Raspberry Pi** para la gestión multimedia y de conectividad.

Los Atmega destacan por su sencillez y amplia documentación, mientras que las Raspberry Pi permiten manejar interfaces gráficas, procesamiento de audio y comuni-

cación con redes externas. Esta combinación asegura una arquitectura balanceada entre nodos ligeros y nodos avanzados.

2.9. Arquitectura distribuida en automoción

La arquitectura distribuida en vehículos implica que cada función relevante es gestionada por una ECU especializada, todas interconectadas por un bus común. Este enfoque modular mejora la escalabilidad, la tolerancia a fallos y la posibilidad de añadir nuevas funciones sin rediseñar completamente el sistema.

Nuestro proyecto se inspira en este principio para añadir dos nuevas unidades: una de confort y otra de multimedia, que se integrarán al bus CAN extendiendo las capacidades del vehículo base.

2.10. Diagnóstico mediante OBD-II

El **On-Board Diagnostics II** (OBD-II) es un estándar internacional (SAE J1962) que permite acceder a datos de diagnóstico del motor y emisiones. Aunque puede emplear diferentes protocolos (ISO 9141, KWP2000, J1850), desde 2008 la normativa exige que todos los vehículos ligeros vendidos en Estados Unidos soporten comunicación CAN en este conector.

En el caso del vehículo de estudio, la única comunicación CAN existente se limita al OBD-II para diagnóstico. Por ello, uno de los objetivos principales del proyecto es extender la red interna para habilitar un bus CAN activo y bidireccional, capaz de integrar nuevas ECUs con funcionalidades ampliadas.

3. Objetivos del proyecto

3.1. Objetivo general

Diseñar e implementar un sistema de automatización distribuido basado en el protocolo *CAN BUS* para integrar nuevas unidades de control en un vehículo antiguo, ampliando sus funcionalidades hacia áreas de confort, multimedia y conectividad avanzada, incluyendo interacción con una aplicación móvil y control por voz mediante inteligencia artificial.

3.2. Objetivos específicos

- Analizar el sistema electrónico original del vehículo y su nivel de conectividad mediante el puerto OBD-II.
- Diseñar e implementar una red CAN adicional que permita la comunicación entre nodos distribuidos.
- Desarrollar una **unidad de confort** encargada de gestionar funciones como climatización, cierre centralizado, control de ventanas y ajustes de iluminación interior.
- Desarrollar una **unidad multimedia** enfocada en la gestión de audio, interfaces de usuario y notificaciones.
- Implementar controladores basados en el chip **MCP2515** y microcontroladores Atmega para asegurar la comunicación estandarizada dentro de la red CAN.
- Incorporar una plataforma de procesamiento avanzado (p. ej., Raspberry Pi) para la gestión de interfaces multimedia, comunicación externa y servicios inteligentes.

- Diseñar e integrar una **aplicación móvil** que permita el monitoreo y control remoto de funciones del vehículo a través de la red CAN.
- Desarrollar un sistema de **control por voz basado en inteligencia artificial**, capaz de ejecutar comandos del usuario para gestionar funciones de confort y multimedia.
- Validar experimentalmente el funcionamiento del sistema mediante pruebas de integración, conectividad móvil y control por voz en condiciones reales.
- Documentar el proceso de desarrollo y generar una bitácora de experimentos que permita futuras mejoras y replicabilidad del sistema.

4. Metodología

La metodología adoptada para el desarrollo del proyecto se estructura en fases progresivas que abarcan desde la investigación inicial hasta la validación experimental del sistema. Cada etapa contempla actividades específicas, herramientas y criterios de evaluación, con el fin de garantizar un proceso de diseño integral, replicable y orientado a resultados.

4.1. Fase 1: Análisis inicial

En esta etapa se llevará a cabo la revisión detallada del vehículo base, incluyendo:

- Inspección del cableado eléctrico existente.
- Identificación de las ECUs presentes y sus funciones.

- Exploración del conector OBD-II y sus protocolos soportados.

El objetivo de esta fase es obtener un diagnóstico general de las capacidades electrónicas del vehículo, sirviendo como base para el diseño de la red CAN extendida.

4.2. Fase 2: Diseño del sistema

Se propondrá la arquitectura de red CAN distribuida a implementar. Las decisiones de diseño incluyen:

- Empleo de **Raspberry Pi** como unidades de control principales (ECUs de confort y multimedia).
- Uso de microcontroladores **Atmega** como nodos secundarios conectados al bus CAN, para la interpretación de mensajes y control de actuadores.
- Definición de sensores y actuadores a integrar (sensores de temperatura, humedad, presión, relés para ventanas, controles de audio).

El resultado esperado de esta fase es un diagrama de bloques de la red CAN propuesta, detallando la distribución de funciones en cada unidad.

4.3. Fase 3: Desarrollo de hardware

Durante esta etapa se desarrollará el hardware requerido, iniciando con módulos comerciales para pruebas y avanzando hacia un diseño de placas personalizadas con el chip **MCP2515**. Las actividades incluyen:

- Montaje de nodos de prueba basados en Arduino + MCP2515.

- Prototipado de la ECU de confort y la ECU multimedia en Raspberry Pi.
- Diseño de una PCB personalizada para la versión final del sistema.

4.4. Fase 4: Desarrollo de software

El software se desarrollará empleando el lenguaje **Rust** en todas las plataformas involucradas, lo que asegura consistencia y eficiencia. Se plantea la siguiente distribución:

- **Nodos (Atmega):** programación en Rust para control de sensores y actuadores, con interpretación de mensajes CAN.
- **ECUs (Raspberry Pi):** aplicaciones en Rust y Tauri para gestión de interfaces, comunicación con la aplicación móvil y servicios inteligentes.
- **Aplicación móvil:** desarrollo en Rust o React Native, con conectividad vía Bluetooth. Inicialmente será compatible con Android y posteriormente con iOS.
- **Control por voz:** integración de librerías locales de reconocimiento de voz (ej. Vosk, PocketSphinx) que permitan ejecutar comandos sin necesidad de servicios en la nube.

4.5. Fase 5: Integración con aplicación móvil

Se desarrollará una aplicación móvil que permita interactuar con el sistema a través de Bluetooth. Entre sus funciones principales estarán:

- Monitoreo de sensores (temperatura, humedad, presión).
- Control remoto de ventanas, climatización y funciones de confort.

- Gestión de audio y multimedia.
- Ejecución de comandos mediante voz, con procesamiento local en la ECU multimedia.

4.6. Fase 6: Validación experimental

La validación se realizará de manera incremental, iniciando con pruebas en banco y posteriormente en el vehículo. Las pruebas incluyen:

- Comunicación CAN entre nodos de prueba.
- Integración de ECUs con sensores y actuadores en entorno controlado.
- Pruebas de conectividad con la aplicación móvil vía Bluetooth.
- Validación del control por voz en condiciones reales dentro del vehículo.

Los resultados de cada prueba se documentarán en una **bitácora de experimentos**, lo que permitirá identificar fallos, corregir iterativamente el diseño y asegurar la replicabilidad del sistema.

5. Diseño y desarrollo del sistema

En esta sección se describe la arquitectura propuesta del sistema, así como el desarrollo de sus componentes de hardware y software. El diseño se basa en una red *CAN BUS* distribuida que conecta las nuevas unidades de control con los nodos electrónicos del vehículo.

5.1. Arquitectura general del sistema

El sistema se concibe bajo una topología de bus CAN, en la cual se interconectan:

- **ECU de confort (Raspberry Pi):** gestiona funciones relacionadas con climatización, ventanas eléctricas, cierre centralizado e iluminación interior.
- **ECU multimedia (Raspberry Pi):** administra el audio, la conectividad con la aplicación móvil, la interfaz gráfica y el control por voz basado en inteligencia artificial.
- **Nodos secundarios (Atmega + MCP2515):** actúan como interfaces entre el bus CAN y los actuadores/sensores, interpretando mensajes y ejecutando órdenes específicas.

La comunicación entre todas las unidades se realiza mediante el protocolo CAN, garantizando compatibilidad y escalabilidad. En la Figura 1 se muestra un diagrama de bloques de la arquitectura propuesta.

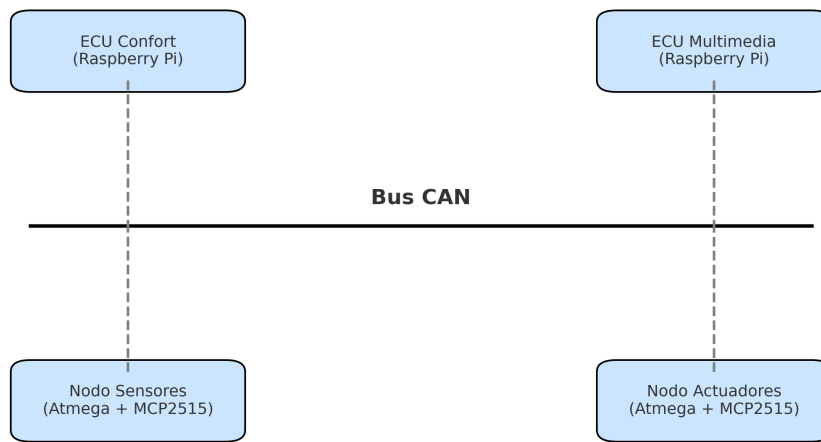


Figura 1: Arquitectura general del sistema de automatización distribuido.

5.2. Diseño de hardware

5.2.1. Módulos de prueba

Para la fase inicial de validación se emplearán módulos comerciales de **MCP2515** conectados a microcontroladores Atmega y Raspberry Pi. Esto permitirá probar la comunicación CAN sin necesidad de diseñar hardware propio.

5.2.2. Placas personalizadas

En la etapa final se diseñarán placas electrónicas que integren:

- Controlador CAN MCP2515.
- Microcontrolador Atmega (para nodos simples).
- Interfaces de conexión a sensores (temperatura, humedad, presión).

- Actuadores (relés para ventanas, salidas digitales para iluminación).

5.3. Diseño de software

5.3.1. ECUs (Raspberry Pi)

Las ECUs estarán programadas en **Rust**, utilizando Tauri para las interfaces gráficas. Las funciones principales incluyen:

- Procesamiento de mensajes CAN entrantes y salientes.
- Control de audio y multimedia.
- Interfaz gráfica para el usuario en pantalla integrada.
- Integración del módulo de control por voz local.
- Comunicación Bluetooth con la aplicación móvil.

5.3.2. Nodos secundarios (Atmega)

Cada nodo estará encargado de un conjunto específico de funciones, implementadas en **Rust embebido**, incluyendo:

- Lectura de sensores (temperatura, humedad, presión).
- Control de actuadores mediante relés y salidas digitales.
- Comunicación CAN mediante MCP2515.

5.3.3. Aplicación móvil

La aplicación móvil se desarrollará en **React Native** con soporte inicial para Android. Sus funciones serán:

- Conexión al vehículo vía Bluetooth.
- Monitoreo de datos en tiempo real (estado de ventanas, clima, sensores).
- Control remoto de funciones de confort y multimedia.
- Envío de comandos de voz hacia la ECU multimedia.

5.4. Control por voz mediante inteligencia artificial

El sistema integrará un motor de reconocimiento de voz local, con librerías como **Vosk** o **PocketSphinx**, que permitirá ejecutar comandos básicos sin conexión a internet. Entre las funciones iniciales se consideran:

- Apertura y cierre de ventanas.
- Activación o desactivación del climatizador.
- Control de reproducción multimedia (play, pausa, siguiente).
- Consulta de datos de sensores (temperatura interior, humedad).

5.5. Flujo de comunicación

El flujo general de datos en el sistema es el siguiente:

1. El usuario interactúa con la aplicación móvil o mediante comandos de voz.

2. La ECU multimedia recibe la orden y la transmite por el bus CAN.
3. El nodo correspondiente interpreta el mensaje y ejecuta la acción sobre el actuador o sensor.
4. El estado actualizado se devuelve por el bus CAN y puede visualizarse en la app o en la interfaz de la ECU multimedia.

Este flujo asegura una integración coherente y escalable de todas las funcionalidades, replicando en un vehículo antiguo las características de los sistemas modernos de automoción conectada.

5.6. Desarrollo

5.6.1. Módulos de temperatura del habitáculo

Para automatizar el climatizador en un futuro, necesitamos poder conocer la temperatura, con cierta precisión, en el interior del vehículo. Actualmente disponemos de varios sensores en el mercado que nos permiten medir la temperatura pero, todos disponen de un margen de error que se arrastra en cada lectura. Con el fin de reducir el error del sensor, se plantea dividir el habitáculo del vehículo en 4 cuadrantes de forma que dispongamos de un sensor por cada cuadrante.

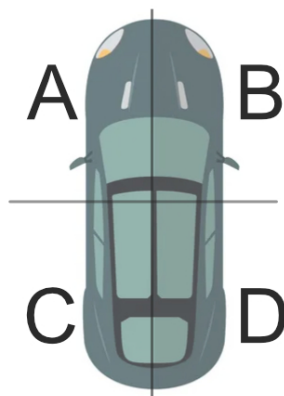


Figura 2: Diagrama de los cuadrantes del vehículo.

Mediante la división en cuadrantes tendremos a nuestra disposición 4 mediciones con las que podremos realizar una media ponderada que será la medición más cercana a la realidad.

Esta media de mediciones se nos hace obligada, en parte, por la definición de temperatura, pues no es más que la energía cinética de un sistema termodinámico, medición que sufre severas alteraciones debido a cualquier radiación directa de infrarojos. *(Esto se puede observar si el sensor se deja al sol directo, la temperatura se disparará pero no será la temperatura del sistema termodinámico que es el habitáculo).*

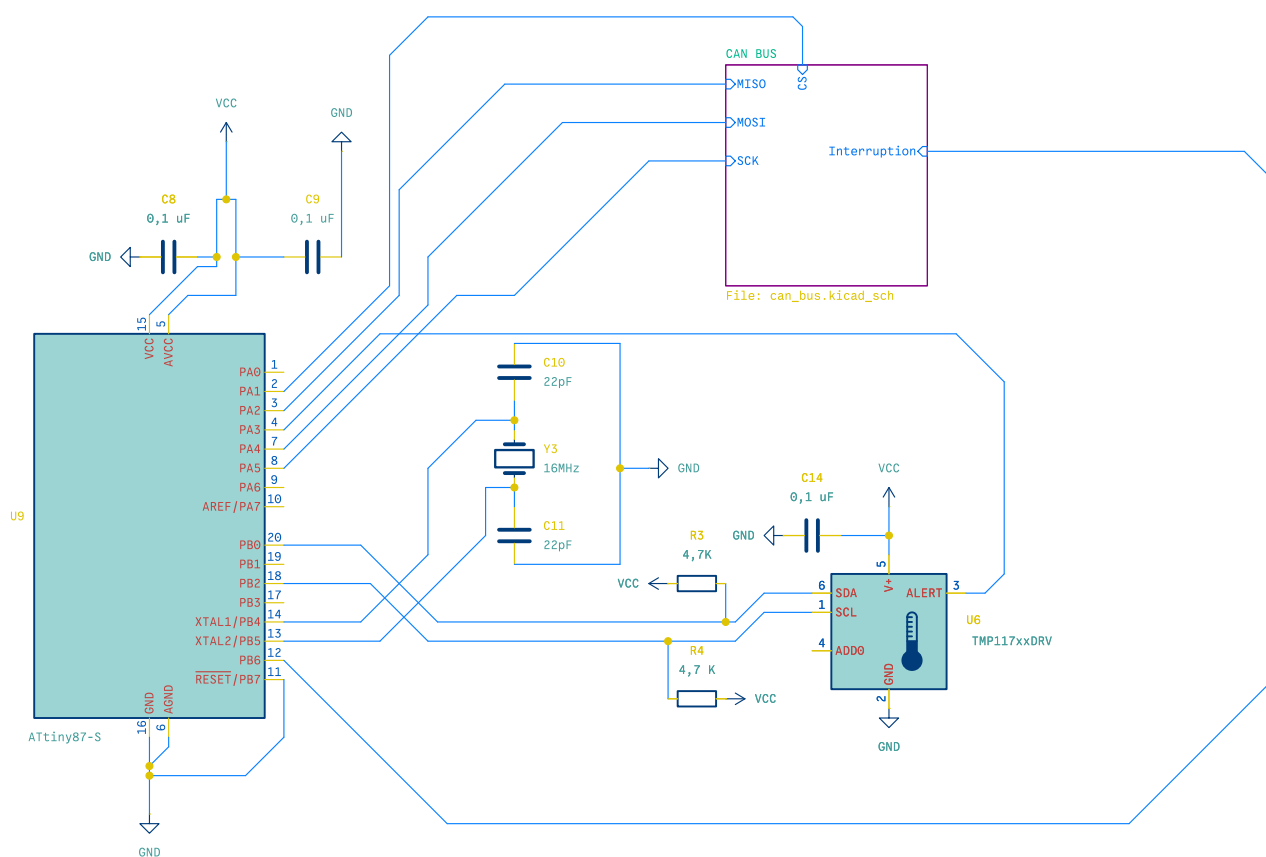
Componentes

Para el desarrollo de los sensores de temperatura usaremos un sensor de grado médico para disponer de mayor precisión, el **TMP117** en su empaquetado SMD. Este sensor nos permite una comunicación I^2C y una precisión de $0,10\text{ }^{\circ}\text{C}$, lo que nos resulta atractivo para poder precisar la temperatura ambiente.

Como cerebro del sistema, durante las pruebas, se plantea usar un **Atmega 328P**, un microcontrolador de 8 bits con soporte para reloj de hasta 16 MHz.

Para el producto final se plantea optimizar el diseño y reducir costes usando un **At-Tiny87** que nos permite usar tanto I^2C como *ISP* para la comunicación entre sensor y transreceptor CAN.

Diagrama del circuito



Unidad de procesamiento y comunicación mediante CAN BUS

Posición: Cuadrante A, B, C, D

Módulo de temperatura del habitáculo

Company: Katalyst - Departamento de Ingeniería

Title: Módulo de Temperatura - TP-0010U001

Size: A4

Date: 2025-08-21

Rev: 001

Author: Marck D. Carrión Guzmán

Id: 2/7

6. Bitácora de Experimentos

A continuación se presenta un registro cronológico de las actividades experimentales realizadas durante el desarrollo del proyecto.